

# WorkAtlas Backend - Systems Architecture Documentation

## Table of Contents

- 1. [System Overview](#)
- 2. [High-Level Architecture](#)
- 3. [Technology Stack](#)
- 4. [Component Structure](#)
- 5. [Data Layer Architecture](#)
- 6. [API Architecture](#)
- 7. [AI/ML Architecture](#)
- 8. [Security Architecture](#)
- 9. [Integration Architecture](#)
- 10. [User Flow Diagrams](#)

## System Overview

WorkAtlas Backend is a high-performance Python backend built with FastAPI, providing REST APIs for an AI-powered job portal. It integrates MongoDB embeddings, and LangChain with OpenAI for intelligent job-seeker matching.

### Key Features

- **FastAPI Framework:** High-performance async API with automatic OpenAPI documentation
- **Dual Database System:** MongoDB for transactional data, ChromaDB for vector search
- **AI-Powered Matching:** LangChain + OpenAI for intelligent candidate-job matching
- **JWT Authentication:** Secure token-based authentication with role-based access
- **Beanie ODM:** MongoDB object-document mapping with Pydantic validation
- **n8n Integration:** Workflow automation for email notifications
- **Vector Embeddings:** OpenAI text-embedding-3-small for semantic search

## High-Level Architecture

 [Mermaid Diagram: Show in external browser](#)

### Architecture Layers

- 1. **API Gateway Layer (FastAPI)**
  - Request routing and validation
  - CORS handling
  - JWT authentication
  - OpenAPI documentation
- 2. **Business Logic Layer (Services)**
  - Seeker operations (registration, profile, applications)
  - Employer operations (jobs, candidates, interviews)

# Technology Stack

## Complete Backend Tech Stack Overview

 [Mermaid Diagram: Show in external browser](#)

## Detailed Technology Breakdown

### 1. Core Framework & Runtime

| Technology | Version | Purpose                                                                 |
|------------|---------|-------------------------------------------------------------------------|
| Python     | 3.12+   | Core programming language with modern features (match/case, type hints) |
| FastAPI    | 0.104+  | High-performance async web framework with automatic OpenAPI docs        |
| Uvicorn    | 0.24+   | Lightning-fast ASGI server with standard support                        |
| Pydantic   | 2.12+   | Data validation and settings management using Python type annotations   |

FastAPI Features Used:

- ✔ Async/await for non-blocking I/O
- ✔ Automatic OpenAPI (Swagger) documentation
- ✔ Request/response validation via Pydantic
- ✔ Dependency injection system
- ✔ Background tasks
- ✔ Middleware support
- ✔ WebSocket support (ready for future use)
- ✔ CORS middleware

Python 3.12 Features:

- ✔ Type hints and type checking
- ✔ Async/await syntax
- ✔ Context managers
- ✔ Pattern matching (match/case)
- ✔ Dataclasses
- ✔ f-strings for formatting

### 2. Database & ODM

| Technology | Version       | Purpose                                               |
|------------|---------------|-------------------------------------------------------|
| MongoDB    | Atlas (Cloud) | NoSQL document database for users, jobs, applications |
| Motor      | 3.7 +         | Async MongoDB driver for Python (used by Beanie)      |

ChromaDB Collections:

- Seeker\_Resume : Resume embeddings for semantic search
- Seeker\_Skills : Skill embeddings for skill matching
- Employer\_JobDescr : Job description embeddings
- Employer\_Skillswish : Required skills embeddings

Embedding Model:

- OpenAI text-embedding-3-small (1536 dimensions)

3. AI & Machine Learning Stack

| Technology       | Version | Purpose                                                        |
|------------------|---------|----------------------------------------------------------------|
| LangChain        | 1.0+    | Framework for building LLM applications with chains and agents |
| LangChain Core   | 1.0+    | Core components for LangChain (prompts, parsers, runnables)    |
| LangChain OpenAI | 0.2+    | OpenAI integration for LangChain                               |
| OpenAI SDK       | 2.8+    | Official OpenAI Python SDK for GPT models and embeddings       |

LangChain Components Used:

-  ChatPromptTemplate: Structured prompts for LLM
-  PydanticOutputParser: Structured output parsing
-  ChatOpenAI: OpenAI model integration
-  Chains: Sequential LLM operations

OpenAI Models:

- GPT-4o-mini: Main model for scoring and reasoning (configurable)
- text-embedding-3-small: Embedding generation (1536 dims, cost-effective)

AI Configuration:

```
# Model Settings
OPENAI_MODEL = "gpt-4o-mini" # Configurable
DEFAULT_TEMPERATURE = 0.75
EMBEDDING_MODEL = "text-embedding-3-small"

# Optional: LangSmith Tracing
LANGSMITH_TRACING = "true"
LANGSMITH_PROJECT = "job_portal"
```

LangSmith Integration (Optional):

- Trace LLM calls for debugging
- Monitor token usage
- Track performance metrics

4. Security & Authentication

```
{
  "sub": "user_id",           # User ID (MongoDB ObjectId)
  "role": "seeker|employer", # User role
  "exp": 1234567890          # Expiration timestamp
}
```

Password Security:

- Minimum length: 6 characters (configurable)
- BCrypt hashing with automatic salt
- 72-byte password limit (BCrypt standard)

5. File Processing

| Technology  | Version | Purpose                                 |
|-------------|---------|-----------------------------------------|
| PyPDF2      | 3.0+    | PDF text extraction for resume parsing  |
| Python-Docx | 1.2+    | DOCX text extraction for resume parsing |
| Aiofiles    | 24.1+   | Async file I/O operations               |

Resume Processing:

- Supported formats: PDF, DOCX, TXT
- Text extraction from documents
- Resume storage in MongoDB
- Embedding generation for ChromaDB

File Size Limits:

- Resume upload: 10MB max (configurable)
- Async file handling for non-blocking uploads

6. HTTP & Integration

| Technology | Version | Purpose                                              |
|------------|---------|------------------------------------------------------|
| HTTPX      | 0.28+   | Modern async HTTP client for external API calls      |
| Requests   | 2.32+   | Synchronous HTTP client (fallback/simple operations) |

HTTPX Features:

-  Async/await support
-  HTTP/2 support
-  Timeout configuration
-  Connection pooling
-  Request/response hooks

| Technology      | Version | Purpose                                |
|-----------------|---------|----------------------------------------|
| Annotated Types | 0.7+    | PEP 593 annotated types for validation |

Loguru Configuration:

```
# Dual log files with rotation
logger.add("Logs/Debug_log.log", level="DEBUG", rotation="50MB")
logger.add("Logs/Error_log.log", level="ERROR", rotation="50MB")
```

Custom Validators:

- US State abbreviations (50 states + DC + 5 territories)
- ZIP codes (5-digit and ZIP+4 formats)
- US phone numbers (multiple formats)
- Email addresses (RFC 5322)
- GLCS industry codes (2, 4, 6, 8 digit formats)

8. Development & Deployment

| Technology     | Version | Purpose                                                       |
|----------------|---------|---------------------------------------------------------------|
| uv             | Latest  | Ultra-fast Python package installer and resolver (Rust-based) |
| Docker         | Latest  | Container platform for consistent deployment                  |
| Docker Compose | Latest  | Multi-container orchestration                                 |

uv Features:

- 10-100x faster than pip
- Automatic dependency resolution
- Lock file generation
- PEP 723 inline script metadata support

Docker Configuration:

```
FROM python:3.12-slim
WORKDIR /app
# Install uv for fast dependency management
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
# Install dependencies
RUN uv sync
# Run application
CMD ["uv", "run", "uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

Docker Compose Services:

- **backend** : FastAPI application (port 8000)
- **frontend** : Nextjs application (port 3000)
- **n8n** : Workflow automation (port 5678)

| Category           | Technologies |
|--------------------|--------------|
| Total Dependencies |              |

Dependency Tree Visualization

🔗 [Mermaid Diagram: Show in external browser](#)

Tech Stack Decision Rationale

Why FastAPI?

- **✅ Performance:** Among the fastest Python frameworks (on par with Node.js)
- **✅ Modern:** Built on async/await, type hints, and Pydantic
- **✅ Auto Documentation:** OpenAPI/Swagger docs generated automatically
- **✅ Type Safety:** Request/response validation via Pydantic
- **✅ Developer Experience:** Excellent IDE support and debugging

Why Beanie ODM?

- **✅ Async Native:** Built for async/await from the ground up
- **✅ Pydantic Integration:** Seamless validation and serialization
- **✅ Type Safety:** Full type hints for MongoDB operations
- **✅ Modern:** More Pythonic than PyMongo or MongoEngine
- **✅ Performance:** Direct Motor integration for speed

Why ChromaDB?

- **✅ Open Source:** No vendor lock-in
- **✅ Easy Setup:** Persistent local storage, no server needed
- **✅ Python Native:** First-class Python support
- **✅ Embedding Functions:** Built-in OpenAI integration
- **✅ Cost Effective:** Self-hosted vector database

Why LangChain?

- **✅ Abstraction:** High-level interface for LLM operations
- **✅ Prompt Management:** Structured prompt templates
- **✅ Output Parsing:** Automatic structured output extraction
- **✅ Tracing:** LangSmith integration for debugging
- **✅ Ecosystem:** Large community and integrations

Why OpenAI?

- **✅ Quality:** State-of-the-art language models
- **✅ Cost Effective:** GPT-4o-mini is affordable
- **✅ Embeddings:** High-quality semantic embeddings
- **✅ Reliability:** Stable API with good uptime
- **✅ Integration:** Excellent LangChain support

|                                  |                                             |
|----------------------------------|---------------------------------------------|
| └─ routes/                       | # Route handlers                            |
| └─ auth_router.py                | # Authentication endpoints                  |
| └─ seeker_router.py              | # Job seeker endpoints                      |
| └─ employer_router.py            | # Employer endpoints                        |
| └─ core/                         | # Core utilities                            |
| └─ security.py                   | # JWT, password hashing                     |
| └─ ai_config.py                  | # LLM configuration                         |
| └─ services/                     | # Business Logic                            |
| └─ seeker_services.py            | # Seeker operations                         |
| └─ employer_services.py          | # Employer operations                       |
| └─ n8n_service.py                | # Email integration                         |
| └─ ai/                           | # AI/ML components                          |
| └─ chains/                       | # LangChain implementations                 |
| └─ matching_service.py           | # Job matching for seekers                  |
| └─ candidate_matching_service.py | # Candidate matching for employers          |
| └─ scoring_chain.py              | # LLM scoring logic                         |
| └─ rag/                          | # Vector operations                         |
| └─ vector_store.py               | # ChromaDB interface                        |
| └─ db/                           | # Database Layer                            |
| └─ settings.py                   | # DB connection config                      |
| └─ seeker_db_ops.py              | # Seeker CRUD operations                    |
| └─ employer_db_ops.py            | # Employer CRUD operations                  |
| └─ chroma_crud_ops.py            | # Vector DB operations                      |
| └─ ingestion/                    | # Data seeding scripts                      |
| └─ ingest_seeker.py              |                                             |
| └─ ingest_employer.py            |                                             |
| └─ ingest_mongo_seeker.py        |                                             |
| └─ ingest_mongo_employer.py      |                                             |
| └─ schemas/                      | # Pydantic models                           |
| └─ seeker.py                     | # Seeker document model                     |
| └─ employer.py                   | # Employer document model                   |
| └─ utils/                        | # Utility functions                         |
| └─ validators.py                 | # Custom validators (address, phone, email) |
| └─ gics_helper.py                | # Industry classification helper            |

## Data Layer Architecture

[🔗 Mermaid Diagram: Show in external browser](#)

## MongoDB Schema

### Seeker Document

|                                |                        |
|--------------------------------|------------------------|
| {                              |                        |
| "_id": ObjectId("..."),        | # MongoDB primary key  |
| "seeker_id": PydanticObjectId, | # MongoDB ObjectId     |
| "seeker_identification": UUID, | # UUID for ChromaDB    |
| "temperature": 0.75,           | # AI model temperature |
| "information": {               | # Personal information |
| "first_name": str,             |                        |
| "last_name": str,              |                        |
| "email": EmailStr,             |                        |
| "phone": USPhoneNumber,        |                        |
| "address": {                   |                        |
| "street": str,                 |                        |
| "city": str,                   |                        |
| "state": StateAlpha            | # Validated US states  |

```

"employer_id": PydanticObjectId,          # MongoDB ObjectId
"employer_identification": UUID,          # UUID for ChromaDB
"company_information": {                  # Company details
    "company_name": str,
    "address": {
        "street": str,
        "city": str,
        "state": StateAbbr,
        "zip_code": ZipCode
    },
    "industry": [                         # Exactly 2 GICS codes
        {
            "code": str,                  # 2, 4, 6, or 8 digits
            "description": str
        }
    ],
    "benefits": str
},
"contact_person": {                      # Primary contact
    "contact_first_name": str,
    "contact_last_name": str,
    "email": EmailStr
},
"password_hash": str,                    # BCrypt hashed password
"created_at": datetime,                  # Account creation
"open_jobs": [                           # Job postings
    {
        "job_id": PydanticObjectId,
        "job_identification": UUID,      # UUID for ChromaDB
        "employer_identification": UUID, # Employer UUID
        "job_title": str,
        "job_description": str,
        "posted_date": datetime,
        "department": str,
        "hire_mgr_first": str,
        "hire_mgr_last": str,
        "current_status": Literal["Posted", "Withdrawn", "Pending", "Canceled"],
        "pay_range": [int, int],
        "pay_unit": Literal["Hourly", "Monthly", "Yearly"],
        "education_level": Literal["BA", "BS", "MA", "MS", "MBA", "PhD"],
        "edu_focus": str,
        "key_skills": List[str]          # Max 15 skills
    }
],
"apps_received": [                       # Applications received
    {
        "applicant_id": str,             # Seeker UUID
        "job_id": str,                   # Job UUID
        "initial_daterec": datetime,
        "current_status": str,
        "previous_status": str | None,
        "current_status_date": datetime
    }
]
}

```

## ChromaDB Schema

### Collection Structure

Each ChromaDB collection stores:

- **ID:** Unique identifier (seeker\_id, job\_id, etc.)
- **Embedding:** 1536-dimensional vector from OpenAI
- **Metadata:** Additional searchable fields

Document Object Text content



```
    "employer_id": str,
    "job_title": str,
    "department": str,
    "education_level": str,
    "posted_date": str
  },
  "document": str          # Job description text
}
```

API Architecture

API Endpoints Overview

 [Mermaid Diagram: Show in external browser](#)

API Endpoint Details

Authentication Endpoints

| Method | Endpoint                       | Description                  |
|--------|--------------------------------|------------------------------|
| POST   | /api/v1/auth/login             | Login and receive JWT token  |
| POST   | /api/v1/auth/register/seeker   | Register new job seeker      |
| POST   | /api/v1/auth/register/employer | Register new employer        |
| GET    | /api/v1/auth/me                | Get current user information |

Seeker Endpoints

| Method | Endpoint                         | Description                  |
|--------|----------------------------------|------------------------------|
| GET    | /api/v1/seekers/me               | Get current seeker profile   |
| PUT    | /api/v1/seekers/me               | Update seeker profile        |
| POST   | /api/v1/seekers/resume           | Upload resume (PDF/DOCX/TXT) |
| GET    | /api/v1/seekers/jobs             | Search available jobs        |
| GET    | /api/v1/seekers/jobs/:id         | Get specific job details     |
| GET    | /api/v1/seekers/recommendations  | Get AI job recommendations   |
| POST   | /api/v1/seekers/applications     | Apply to a job               |
| GET    | /api/v1/seekers/applications     | Get all applications         |
| DELETE | /api/v1/seekers/applications/:id | Withdraw application         |

| Method | Endpoint                                                      | Description                    |
|--------|---------------------------------------------------------------|--------------------------------|
| POST   | /api/v1/employers/jobs/:id/candidates/:sid/schedule-interview | Schedule interview             |
| GET    | /api/v1/employers/applications                                | Get all applications for a job |
| GET    | /api/v1/employers/n8n/status                                  | Check n8n service status       |

Request/Response Flow

[🔗 Mermaid Diagram: Show in external browser](#)

AI/ML Architecture

AI Matching Pipeline

[🔗 Mermaid Diagram: Show in external browser](#)

LLM Scoring Chain

[🔗 Mermaid Diagram: Show in external browser](#)

Scoring Weights

```
# Overall Score Calculation (Weighted Average)
overall_score = (
    skills_score * 0.40 +      # 40% weight - most important
    education_score * 0.20 +   # 20% weight
    pay_score * 0.20 +         # 20% weight
    experience_score * 0.20    # 20% weight
)
```

Match Score Interpretation

| Score Range | Match Quality | Description                            |
|-------------|---------------|----------------------------------------|
| 0-40        | Poor          | Significant mismatches in requirements |
| 41-70       | Moderate      | Some alignment but notable gaps        |
| 71-85       | Good          | Strong alignment with minor gaps       |
| 86-100      | Excellent     | Exceptional match, highly recommended  |

Security Architecture

Authentication Flow

- **Storage:** localStorage on frontend

3. Input Validation

- **Email:** RFC 5322 compliant validation
- **Phone:** Google libphonenumber validation
- **Address:** US state/ZIP code validation
- **GICS Codes:** Industry code validation

4. API Security

- **CORS:** Whitelist of allowed origins
- **Rate Limiting:** (Ready for implementation)
- **HTTPS:** Production deployment requirement
- **Token Refresh:** (Ready for implementation)

---

## Integration Architecture

### n8n Workflow Integration

 [Mermaid Diagram: Show in external browser](#)

### Interview Scheduling Flow

 [Mermaid Diagram: Show in external browser](#)

### Email Template Data

```
{
  "seeker_email": "john@example.com",
  "job_title": "Senior Software Engineer",
  "employer_name": "Tech Company Inc.",
  "employer_email": "hr@techcompany.com",
  "interview_date": "December 15, 2024",
  "interview_time": "2:00 PM",
  "interview_type": "Video", # In-person | Video | Phone
  "location_or_link": "https://meet.google.com/abc-defg-hij",
  "notes": "Please bring your portfolio"
}
```

---

## User Flow Diagrams

### 1. Job Seeker Registration & Recommendations Flow

 [Mermaid Diagram: Show in external browser](#)

### 2. Employer Job Posting & Candidate Matching Flow

 [Mermaid Diagram: Show in external browser](#)

### 3. AI Matching Pipeline Detailed Flow

3. API Optimization

- **Async Operations:** Non-blocking I/O throughout stack
- **Response Streaming:** (Ready for large result sets)
- **Compression:** Gzip response compression
- **Caching:** (Ready for Redis implementation)

4. Scalability

- **Stateless API:** Easy horizontal scaling
- **Database Sharding:** MongoDB Atlas auto-sharding
- **Load Balancing:** Ready for multi-instance deployment
- **Microservices:** Modular architecture for service separation

Deployment Architecture

 [Mermaid Diagram: Show in external browser](#)

Environment Variables

```
# MongoDB Configuration
MONGO_DB_USER=test_db_user
MONGO_DB_PASSWORD=<secret>
MONGO_DB_URL=mongodb+srv://${MONGO_DB_USER}:${MONGO_DB_PASSWORD}@cluster.mongodb.net/?appName=job-portal
MONGO_DB_NAME=job-portal

# ChromaDB Configuration
CHROMA_PC_PATH=./chroma_db

# OpenAI Configuration
OPENAI_API_KEY=<secret>
OPENAI_MODEL=gpt-4o-mini

# LangSmith Configuration (Optional)
LANGSMITH_API_KEY=<secret>
LANGSMITH_TRACING=true
LANGSMITH_PROJECT=job_portal

# n8n Configuration
N8N_WEBHOOK_URL=http://n8n:5678/webhook/interview-schedule

# CORS Configuration
CORS_ORIGINS=http://localhost:3000
ALLOW_ORIGINS=http://localhost:3000
```

Error Handling & Logging

Logging Strategy

 [Mermaid Diagram: Show in external browser](#)

Error Response Format



- Fine-tuned models for domain-specific matching
- Resume parsing with AI
- Interview question generation
- Candidate skill gap analysis

#### 4. Security Enhancements

- OAuth2 provider integration
- Two-factor authentication
- API rate limiting with Redis
- Token refresh mechanism

#### 5. Monitoring & Observability

- Prometheus metrics
- Grafana dashboards
- Error tracking (Sentry)
- Performance monitoring (New Relic)

---

## Conclusion

The WorkAtlas backend is a modern, scalable, and AI-powered job portal API built with FastAPI, MongoDB, ChromaDB, and LangChain. It provides:

- **High Performance:** Async operations throughout the stack
- **Type Safety:** Pydantic validation and Python type hints
- **AI-Powered:** LLM-based matching with detailed reasoning
- **Secure:** JWT authentication, BCrypt hashing, input validation
- **Scalable:** Stateless architecture ready for horizontal scaling
- **Developer Friendly:** Auto-generated docs, structured code, comprehensive logging

The modular architecture and clean separation of concerns provide a solid foundation for future enhancements and scalability.

---

**Document Version:** 1.0

**Last Updated:** November 17, 2025

**Author:** System Architecture Analysis

**Project:** WorkAtlas Job Portal Backend